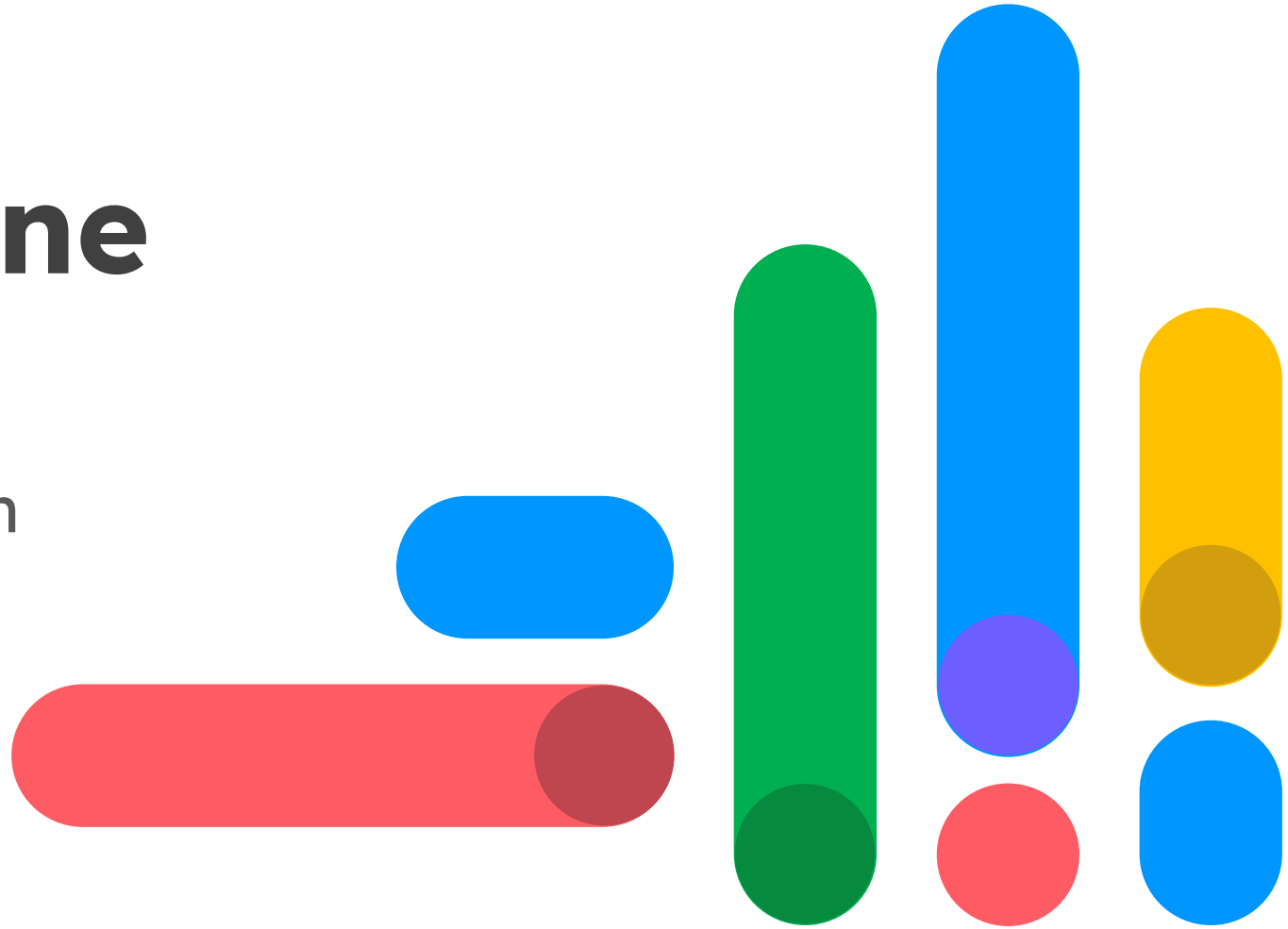
Deep Learning

Applied Machine Learning

Computer Visions :: Detection

Assistant Professor | **Chun-Hsiang Chan**

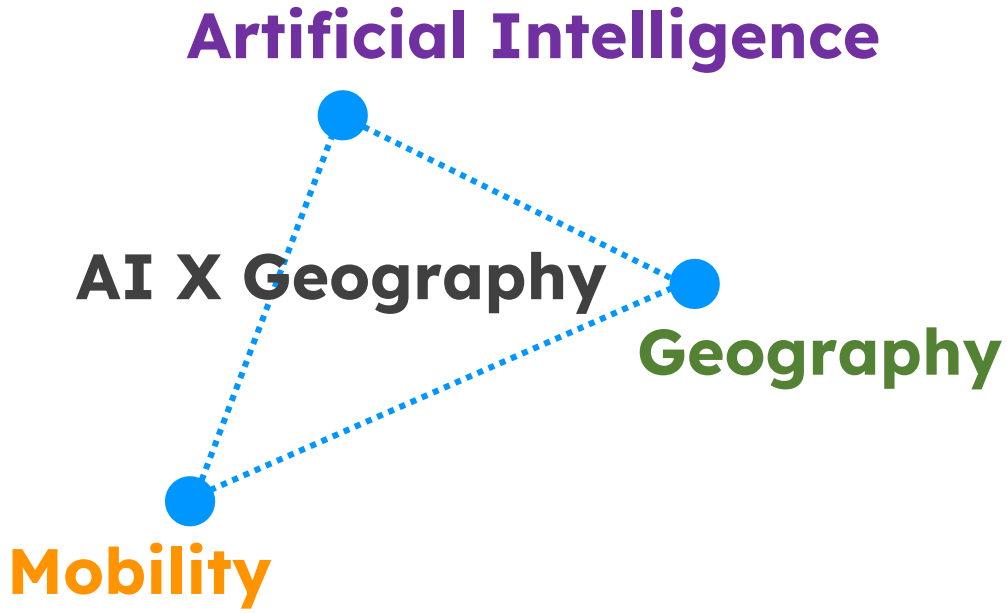
Department of Geography
National Taiwan Normal University



Outline

#GeoAI #Mobility #Geography #STDS

01 Classification, Detection, & Segmentation
02 Bounding Box
03 Intersection over Union
04 Non-Maximum Suppression
05 R-CNN Family
06 You Only Look Once
07 References



Classification, Detection, & Segmentation

The fundamental differences between these tasks concern how much spatial information the model must provide and whether its predictions describe an entire image, individual objects, or individual pixels.

Suppose a room image contains two cats and one dog.



Classification, Detection, & Segmentation

Classification

Image classification determines **which categories** apply to an **entire** image. Basic single-label classification assumes that each image has one correct category:

$$f_\theta(I) \rightarrow p$$



where:

I is the input image.

f_θ is a model with parameters θ.

p contains the predicted probabilities of the categories.

Classification, Detection, & Segmentation

Classification

For example:

$$\boldsymbol{p} = [0.05, 0.90, 0.05]$$

If the category order is “indoor, street scene, forest,” the model selects “street scene.”

However, classification does not always produce only one label. **Multi-label classification** can simultaneously predict “contains people,” “contains cars,” and “contains a road.”

Classification, Detection, & Segmentation

Outputs of Classification

Classification Settings	Relationship between Categories	Common Output Function
Single-label Multiclass classification	Categories are mutually exclusive	Softmax
Multi-label classification	Categories can coexist	An independent sigmoid for each category

Classification, Detection, & Segmentation

Detection

What objects are present, and **where** is each object?

A typical output is a set of object predictions:

$$\mathcal{D} = \{(b_i, c_i, s_i)\}_{i=1}^{N}$$

where:

b_i is the bounding box of object i

c_i is its category

s_i is its confidence score

N is the number of predicted objects

Cat score 0.96, bounding box A
Cat score 0.91, bounding box B
Cat score 0.88, bounding box C
Dog score 0.97, bounding box D
Dog score 0.94, bounding box E

Classification, Detection, & Segmentation

Detection

Detection can therefore support object counting, localization, and subsequent tracking.

However, a bounding box provides only a rectangular region. A pedestrian's box includes background around the body and does not precisely describe the outlines of the arms, legs, or hair.

Classification, Detection, & Segmentation

Segmentation

Segmentation produces predictions at the **pixel level**.

Which category or **individual** object does each pixel belong to?

Two common types are:

Semantic segmentation: distinguishes the category of each pixel.

Instance segmentation: additionally distinguishes different individuals within the same category.

Classification, Detection, & Segmentation

Task	Output unit	Identifies categories	Provides locations	Describes pixel-level shapes	Separates objects of the same category
Classification	Entire image	Yes	Usually no	No	No
Object detection	Each object's box	Yes	Yes	No	Yes
Semantic segmentation	Each pixel	Yes	Yes	Yes	No
Instance segmentation	Each object's mask	Yes	Yes	Yes	Yes

Bounding Box

Bounding Box

A bounding box is a geometric enclosure around an object. The most common type is a rectangle aligned with the image coordinate axes.

It represents an object's location and size using only a few numerical values and is a fundamental data structure in object detection.

Bounding Box

Bounding Box :: Representation

1) Two corners
   $\boldsymbol{b} = (x_{min}, y_{min}, x_{max}, y_{max})$
2) Top-left corner,
   width, & height
   $b = (x_{min}, y_{min}, w, h)$
3) Center, width, & height
   $\boldsymbol{b} = (c_x, c_y, w, h)$



Bounding Box

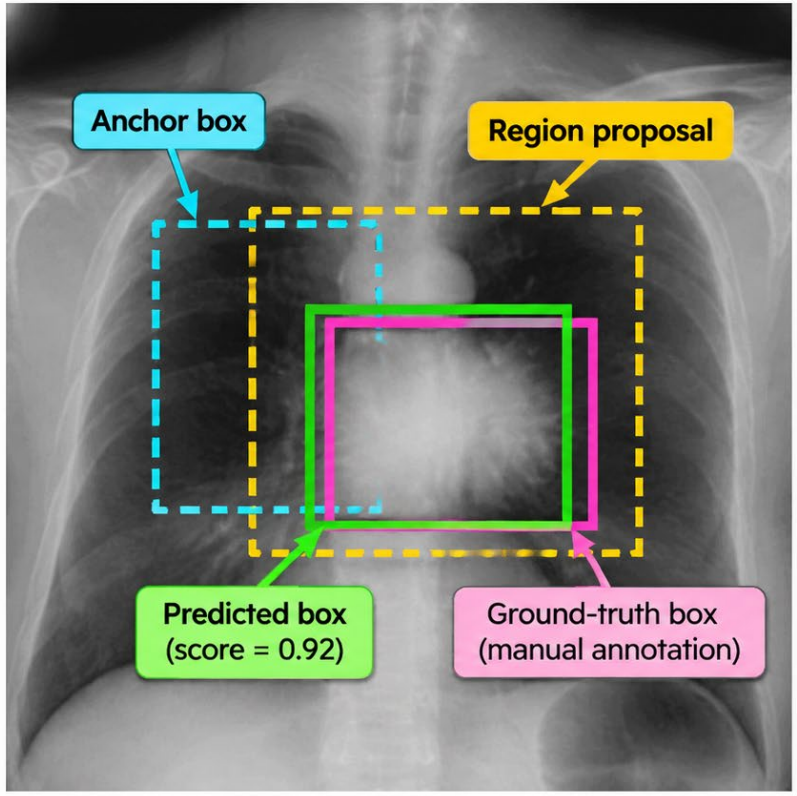
Bounding Box :: Normalization

In reality, the size and resolution of each image are varying since the model cannot transferable for other image sizes or resolutions.

Therefore, we need to do normalization (0~1) for defining the location information of bounding boxes of each images, which facilitate the balance of loss function (classification loss (based on probability 0~1) and bounding box regression loss [location bias]), gradient stability, resolution independence, and data augmentation.

Bounding Box

Bounding Box :: Normalization



Anchor box — A reference box established in advance or generated according to predefined rules

Region proposal — A candidate region that may contain an object

Predicted box — A box produced by the model

Ground-truth box — A manually annotated reference box

Relationship: The predicted box aims to match the ground-truth box, and both are usually generated from a region proposal, which often stems from anchor boxes.

Bounding Box

Bounding Box :: Bounding Box Regression

A model must learn not only that “a car is here,” but also how to adjust the box’s position and size.

Suppose a reference box has center and dimensions:

$$\left(c_x^a, c_y^a, w_a, h_a\right)$$

And the target box is

$$\left(c_x^*, c_y^*, w_a, h_a\right)$$

A common regression target is

$$t_x^* = \frac{c_x^* - c_x^a}{w_a}; t_y^* = \frac{c_y^* - c_y^a}{h_a}; t_w^* = log\frac{w^*}{w_a}; t_h^* = log\frac{h^*}{h_a}$$

The model predicts how far the reference box should move and how much it should change in size.

Bounding Box

Bounding Box :: Potential Issues

Rectangles may be inefficient representations for elongated, tilted, or irregularly shaped objects. For example, a horizontal box around a diagonally oriented ship may contain a large amount of water.

When images are resized, cropped, flipped, or padded, their annotations must be transformed consistently. Incorrect coordinate transformations are a common cause of object detection training problems.



Intersection over Union (IoU)

Intersection over Union (IoU)

IoU measures the overlap between two regions:

$$IoU(A,B) = \frac{|A \cap B|}{|A \cup B|}, 0 \leq IoU \leq 1$$

$$IoU(A,B) = \frac{Area\ of\ overlap}{Total\ area\ covered\ by\ either\ region}$$

$IoU = 0$ the regions do not overlap

$IoU = 1$ the regions are identical

Intersection over Union (IoU)

Intersection over Union (IoU) :: Usage

Evaluating localization quality: comparing a prediction with ground truth.
Assigning training samples: deciding which ground-truth object an anchor or proposal should match.
Removing duplicate predictions: determining whether two predicted boxes may refer to the same object.

Intersection over Union (IoU)

Intersection over Union (IoU) :: IoU as a training loss

An IoU loss can be defined as:

$$L_{IoU} = 1 - IoU$$

However, when two boxes do not overlap, ordinary IoU does not provide useful positional gradients within the non-overlapping region.

The model may therefore receive little guidance about which direction the box should move.

Intersection over Union (IoU)

Intersection over Union (IoU) :: IoU as a training loss

GIoU introduces the smallest enclosing box (C):

$$GIoU = IoU - \frac{|C \backslash (A \cup B)|}{|C|}$$

The second term penalizes the uncovered space within the enclosing box. It therefore provides additional geometric information even when the boxes do not overlap.

Unlike ordinary IoU, GIoU can be negative.

Intersection over Union (IoU)

Intersection over Union (IoU) :: IoU as a training loss

Why are small objects more sensitive?

A shift of a few pixels may have little effect on a large box. For an object only a few pixels wide, the same shift can dramatically reduce the overlap.

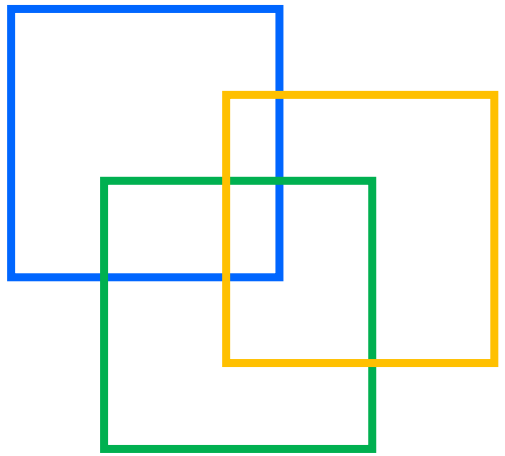
Small-object detection is therefore sensitive not only to recognition difficulty, but also to localization and annotation errors.

Non-Maximum Suppression (NMS)

Non-Maximum Suppression (NMS)

A detector often predicts several similar boxes for the same object. These predictions may come from neighboring locations, different anchors, or feature maps at different scales.

NMS aims to **retain more confident predictions and remove highly redundant boxes**.



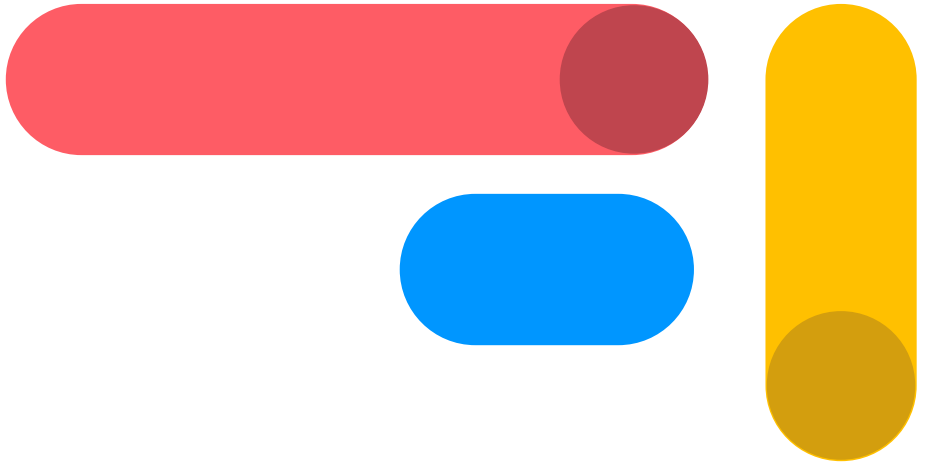
Non-Maximum Suppression (NMS)

Non-Maximum Suppression (NMS) :: Standard greedy NMS

1) For a set of candidate boxes belonging to the same category:
2) Sort the boxes by score in descending order.
3) Select the highest-scoring box and add it to the retained results.
4) Compute the IoU between that box and the remaining boxes.
5) Remove boxes whose IoU exceeds the threshold.
6) Repeat the process with the remaining boxes.

This is a greedy algorithm: each iteration retains the highest-scoring currently available box.

Non-Maximum Suppression (NMS)



Non-Maximum Suppression (NMS)

How does the NMS threshold affect results?

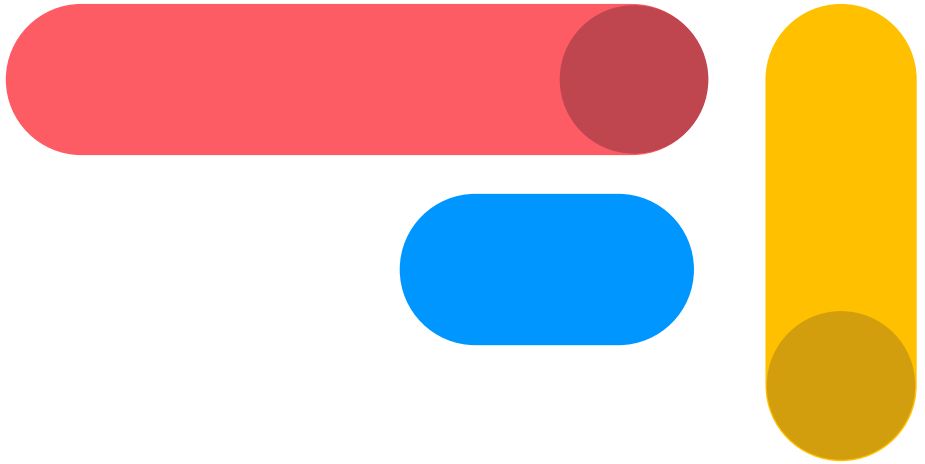
A **lower** threshold **removes** overlapping boxes more aggressively.

A **higher** threshold **allows** more overlapping boxes to remain.

A **lower** threshold is **not** necessarily **better**.

In a crowded scene, the boxes of two different people can overlap substantially. Aggressive suppression may incorrectly remove the second person.

Non-Maximum Suppression (NMS)



Non-Maximum Suppression (NMS)

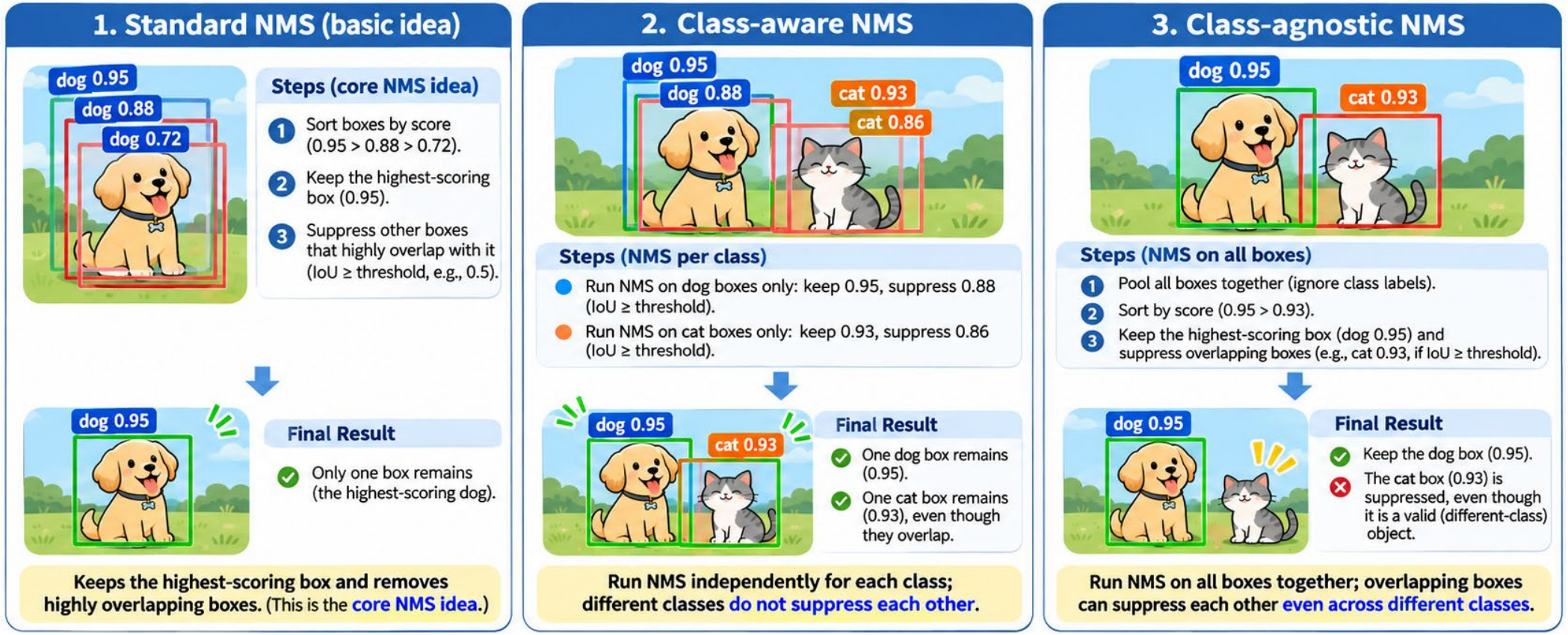
Class-aware NMS generally processes each category separately.

For example, overlapping “person” and “bicycle” boxes do not suppress one another merely because they overlap.

Class-agnostic NMS processes boxes together without separating them by category. This may suit certain applications, but it can also suppress objects from different categories that genuinely coexist.

Non-Maximum Suppression (NMS)

Non-Maximum Suppression (NMS)



Non-Maximum Suppression (NMS)

Non-Maximum Suppression (NMS)

Threshold	What it compares	Purpose
Confidence threshold	Prediction score	Removes low-scoring predictions
NMS IoU threshold	Prediction against prediction	Removes duplicate predictions
Evaluation IoU threshold	Prediction against ground truth	Determines whether a detection is correct

Non-Maximum Suppression (NMS)

Non-Maximum Suppression (NMS) :: Limitations

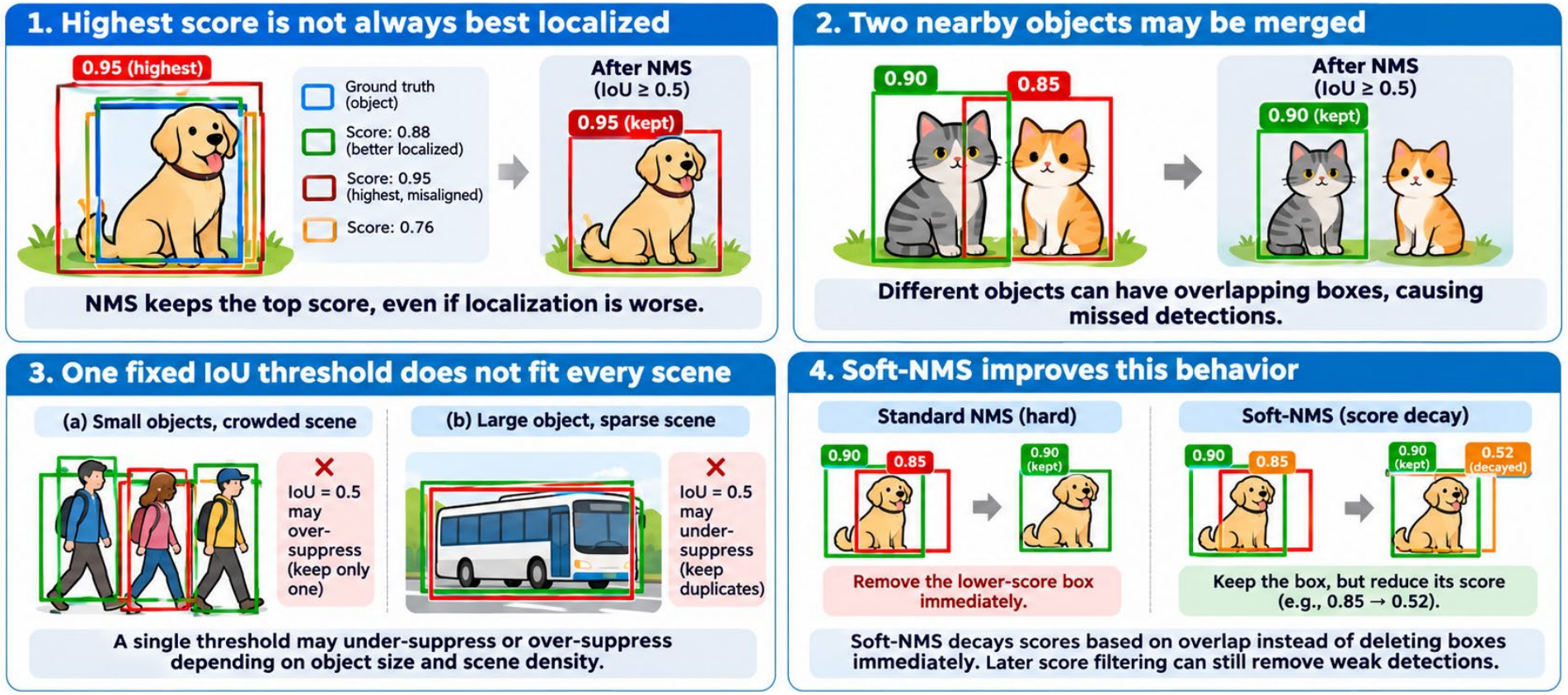
The highest-scoring box is not necessarily the most accurately localized. Different objects may have highly overlapping boxes.

One fixed threshold may not suit every object size or scene.

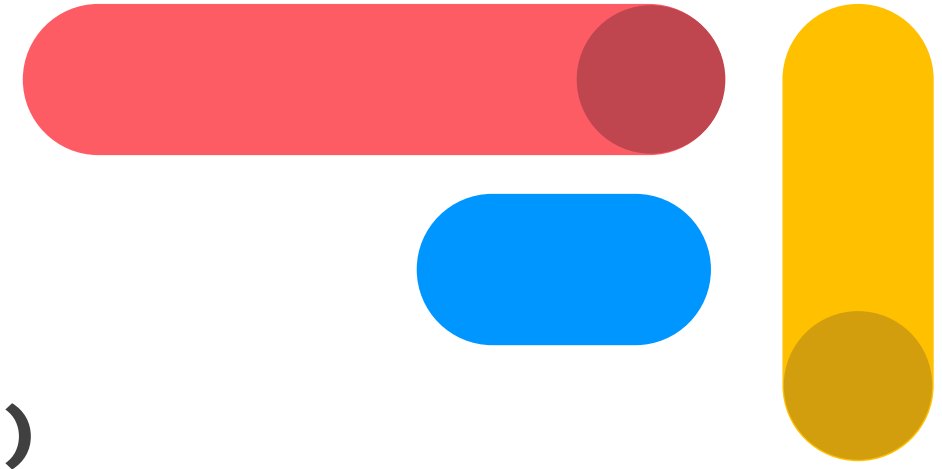
Greedy local decisions do not fully capture relationships between objects. Soft-NMS reduces scores according to overlap instead of immediately removing every highly overlapping box. Later score filtering can still remove low-scoring results

R-CNN Family

Non-Maximum Suppression (NMS) :: Limitations



R-CNN Family



Regions with CNN features (R-CNN)

The R-CNN family provides a useful way to understand how object detectors progressively addressed computational bottlenecks.

The main progression is from

$$R - CNN \Rightarrow Fast\ R - CNN \Rightarrow Faster\ R - CNN$$

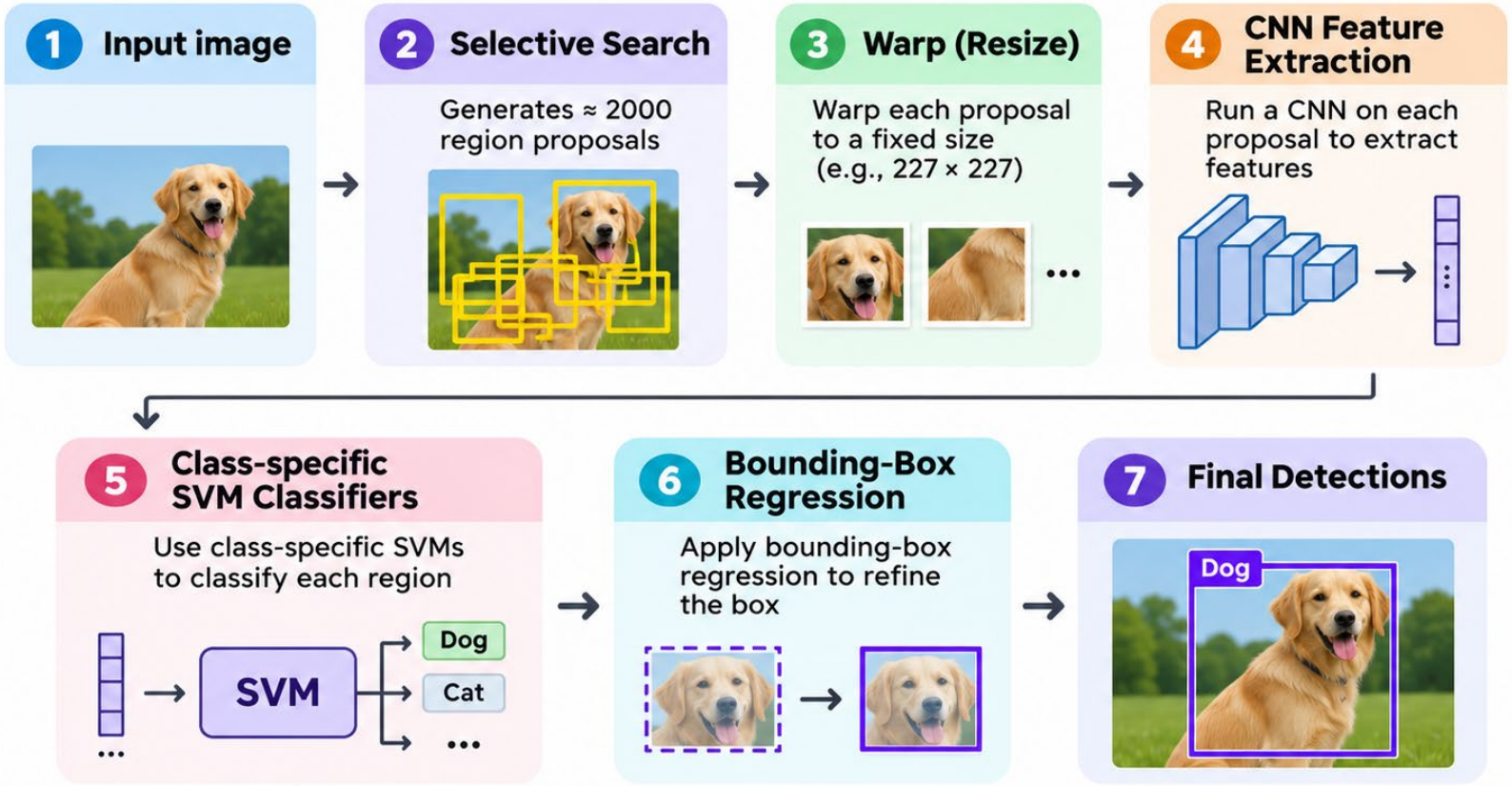
Mask R-CNN subsequently extended this architectural family to instance segmentation.

R-CNN Family

Regions with CNN features (R-CNN) :: R-CNN

The Basic R-CNN pipeline

Its major contribution was the effective use of CNN representations for object detection.
The difficulty is that many proposals overlap heavily, yet their CNN features are computed separately.

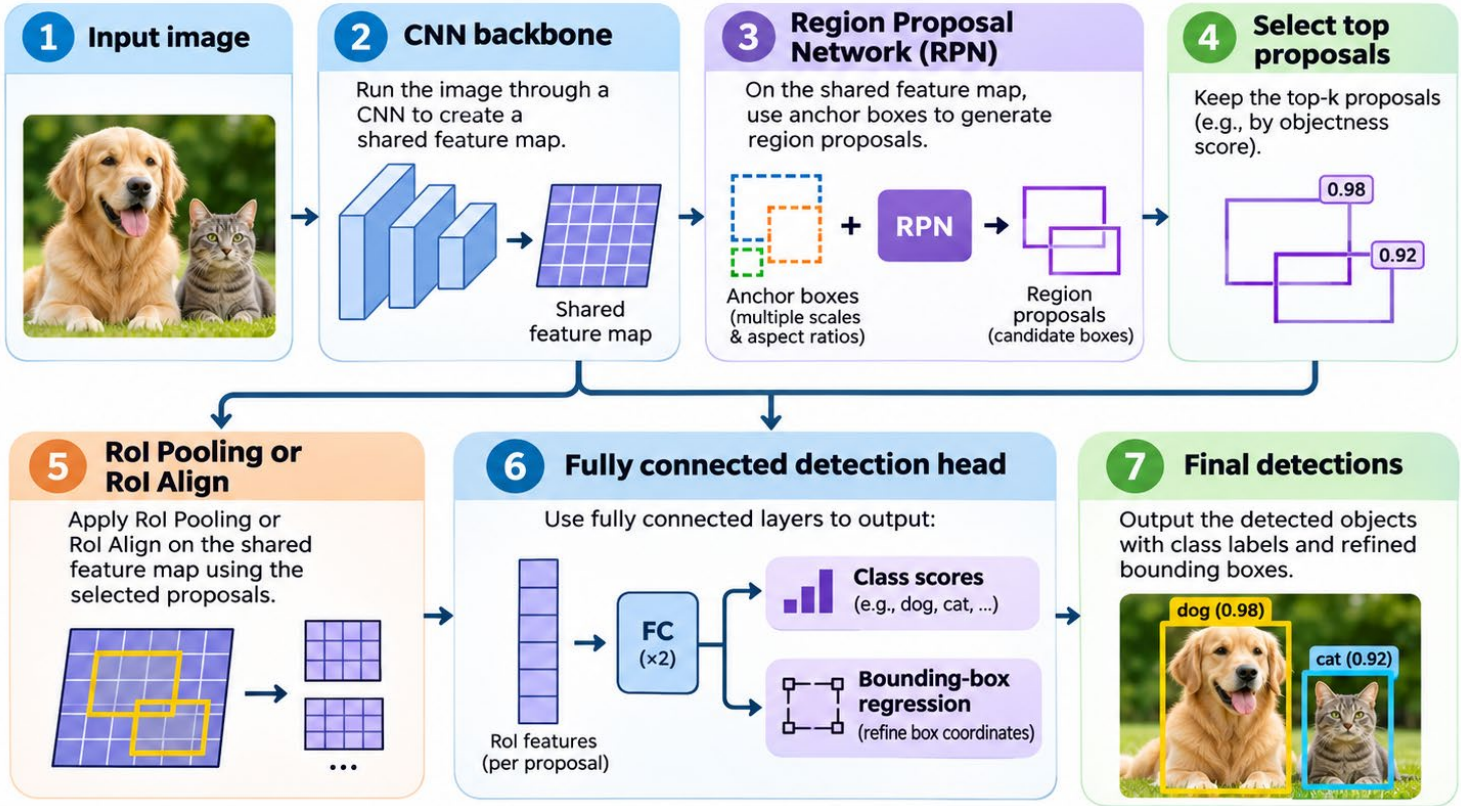


R-CNN Family

Regions with CNN features (R-CNN) :: R-CNN

The Fast R-CNN pipeline

Proposals still come from an external method, but each region now obtains information from a shared feature map.
The key operation is **RoI Pooling**. RoI stands for Region of Interest. Because proposals have different sizes, RoI Pooling converts each region into a fixed-size feature representation for subsequent processing.



R-CNN Family

Regions with CNN features (R-CNN) :: Fast R-CNN

Fast R-CNN also jointly learns classification and box regression within one network:

$$L = L_{cls} + \lambda L_{box}$$

The box regression term is generally computed only for foreground RoIs.

Repeated CNN computation is greatly reduced, but generating external proposals can still be a bottleneck.

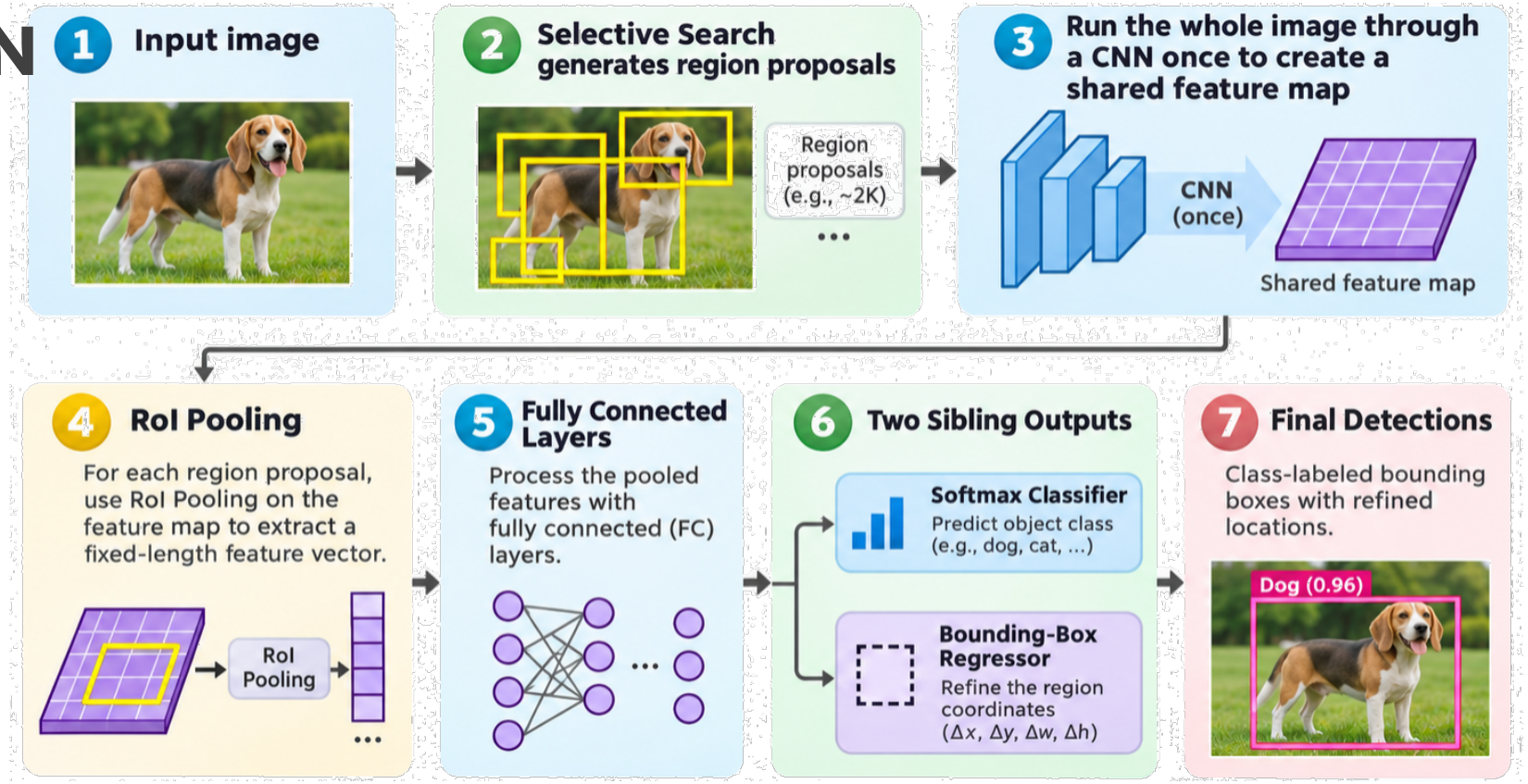
R-CNN Family

Regions with CNN features (R-CNN) :: Faster R-CNN

The Faster R-CNN pipeline

At locations on the feature map, the Region Proposal Network (RPN) predicts:

- Whether a reference box may contain an object: objectness.
- How the reference box should be adjusted.



R-CNN Family

Regions with CNN features (R-CNN) :: Faster R-CNN

RPN objectness generally distinguishes “object” from “background.” It does not yet perform the final classification into categories such as person, car, or dog.

The second stage performs more detailed classification and localization refinement. The RPN and detection network can share convolutional features.

R-CNN Family

Regions with CNN features (R-CNN) :: Faster R-CNN

Method	Proposal source	CNN feature extraction	Main improvement
R-CNN	Selective Search	Separately for each region	Introduces CNN features into detection
Fast R-CNN	External proposals	Shared across the image	Reduces repeated feature computation
Faster R-CNN	RPN	Shared across the image	Learns proposal generation

You Only Look Once (YOLO)

You Only Look Once (YOLO)

YOLOv1: organizing predictions with a grid

The original YOLO divides the image into an:

$$S \times S \text{ grids}$$

The cell containing an object's center is responsible for predicting that object.

Each cell predicts: B boxes; four coordinates and one confidence score for each box; C conditional class probabilities.

You Only Look Once (YOLO)

You Only Look Once (YOLO)

The output shape is therefore: $S \times S \times (5B + C)$ grids

Note that YOLOv1 shares one set of class predictions within each cell; each box does not have a fully independent class vector.

The original box confidence definition combines object presence and localization quality:

$$P(Object) \times IoU$$

During inference, the network predicts a learned estimate of this score. It does not receive ground-truth boxes and calculate the actual IoU.

You Only Look Once (YOLO)

You Only Look Once (YOLO)

Three common components in later YOLO architectures

Component	Function
Backbone	Extracts image features
Neck	Combines features at different scales
Head	Predicts categories, locations, and other outputs

You Only Look Once (YOLO)

You Only Look Once (YOLO)

Version	Year	Core Architecture & Technologies	Key Innovations & Features	Main Limitations
YOLOv1	2016	Grid-based Regression	Introduced real-time object detection using a single neural network; formulated detection as a regression problem.	Poor performance on small and crowded objects; limited localization accuracy.
YOLOv2	2017	Darknet-19, Anchor Boxes, Batch Normalization	Introduced anchor boxes and dimension clustering to improve localization and recall.	Limited multi-scale feature representation.
YOLOv3	2018	Darknet-53, FPN, Multi-scale Prediction	Introduced predictions at three feature-map scales, improving small-object detection.	Increased architectural complexity; relies on anchors and NMS.
YOLOv4	2020	CSPDarknet53, SPP, PANet, Mosaic Augmentation	Integrated architectural improvements and advanced training techniques to balance speed and accuracy.	More complex training configurations.

Note: YOLO versions were developed by different research groups and do not represent a single, strictly sequential architectural lineage. The table summarizes their principal contributions rather than directly comparable benchmark results.

You Only Look Once (YOLO)

You Only Look Once (YOLO)

Version	Year	Core Architecture & Technologies	Key Innovations & Features	Main Limitations
YOLOv5	2020	PyTorch, CSP, AutoAnchor	Simplified model training and deployment; introduced scalable model variants.	Primarily anchor-based; requires NMS.
YOLOv6	2022	EfficientRep, Rep-PAN, Anchor-free Detection	Optimized inference efficiency and hardware deployment for industrial applications.	Performance gains depend on hardware and deployment settings.
YOLOv7	2022	E-ELAN, Trainable Bag-of-Freebies	Improved gradient propagation and training efficiency while maintaining strong speed–accuracy trade-offs.	Complex architecture and optimization strategies.
YOLOv8	2023	C2f, Anchor-free, Decoupled Head	Adopted anchor-free detection and expanded support for multiple computer vision tasks.	Standard detection pipelines still require NMS.

Note: YOLO versions were developed by different research groups and do not represent a single, strictly sequential architectural lineage. The table summarizes their principal contributions rather than directly comparable benchmark results.

You Only Look Once (YOLO)

You Only Look Once (YOLO)

Version	Year	Core Architecture & Technologies	Key Innovations & Features	Main Limitations
YOLOv9	2024	PGI, GELAN	Introduced Programmable Gradient Information to improve gradient flow and reduce information loss.	More sophisticated training mechanisms.
YOLOv10	2024	Consistent Dual Assignments, End-to-End Detection	Introduced NMS-free end-to-end object detection, reducing post-processing latency.	Requires specialized label assignment and training strategies.
YOLO11	2024	C3k2, C2PSA Attention	Enhanced feature extraction and computational efficiency through architectural refinements.	Conventional detection inference still uses NMS.
YOLOv12	2025	Area Attention, R-ELAN	Introduced attention-centric architecture to improve feature representation and contextual modeling.	Potentially higher memory requirements and training complexity.

Note: YOLO versions were developed by different research groups and do not represent a single, strictly sequential architectural lineage. The table summarizes their principal contributions rather than directly comparable benchmark results.

You Only Look Once (YOLO)

You Only Look Once (YOLO)

Version	Year	Core Architecture & Technologies	Key Innovations & Features	Main Limitations
YOLOv13	2025	HyperACE, FullPAD, Hypergraph Modeling	Incorporated hypergraph-based high-order feature relationships for improved feature aggregation.	Increased architectural complexity and optimization requirements.
YOLO26	2026	NMS-free Head, DFL-free, Edge Optimization	Simplified inference and deployment through NMS-free prediction capabilities and edge-oriented optimization.	NMS-free inference is configuration-dependent; not all inference paths eliminate NMS.

Note: YOLO versions were developed by different research groups and do not represent a single, strictly sequential architectural lineage. The table summarizes their principal contributions rather than directly comparable benchmark results.

You Only Look Once (YOLO)

You Only Look Once (YOLO)

Object Detection

https://huggingface.co/spaces/Ultralytics/YOLO26

Webcam Object Detection

https://huggingface.co/spaces/indersingh17188/live-yolo-webcam-demo

References

1) R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 580–587.
2) R. Girshick, “Fast R-CNN,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2015.
3) S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
4) J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
5) W. Liu *et al.*, “SSD: Single shot multibox detector,” in *Proc. European Conf. Computer Vision (ECCV)*, 2016, pp. 21–37.
6) T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2980–2988.

References

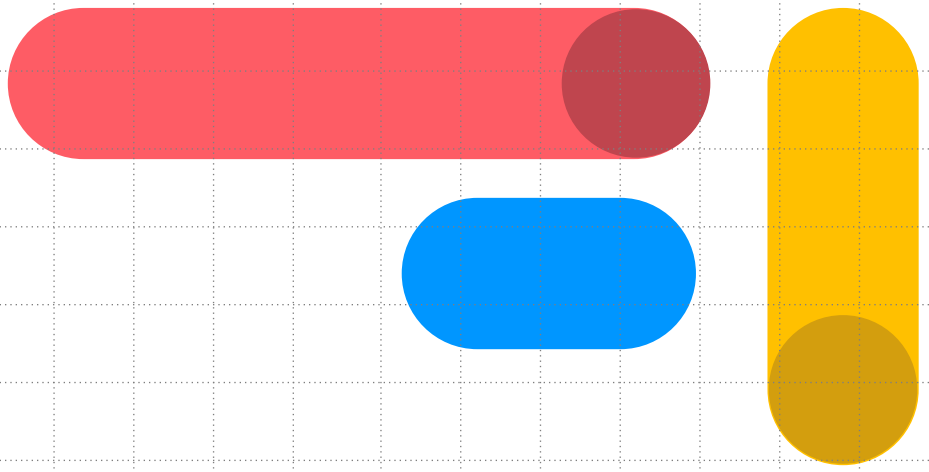
7) H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid, and S. Savarese, “Generalized intersection over union: A metric and a loss for bounding box regression,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019.
8) N. Bodla, B. Singh, R. Chellappa, and L. S. Davis, “Soft-NMS—Improving object detection with one line of code,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017.
9) J. Long, E. Shelhamer, and T. Darrell, “Fully convolutional networks for semantic segmentation,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3431–3440.
10) O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Proc. Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
11) V. Badrinarayanan, A. Kendall, and R. Cipolla, “SegNet: A deep convolutional encoder-decoder architecture for image segmentation,” *arXiv preprint arXiv:1511.00561*, 2015.

References

12) K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
13) K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2961–2969.
14) J. Redmon and A. Farhadi, “YOLO9000: Better, faster, stronger,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017.
15) J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement,” *arXiv preprint arXiv:1804.02767*, 2018.
16) A. Wang *et al.*, “YOLOv10: Real-time end-to-end object detection,” *arXiv preprint arXiv:2405.14458*, 2024.
17) F. Milletari, N. Navab, and S.-A. Ahmadi, “V-Net: Fully convolutional neural networks for volumetric medical image segmentation,” in *Proc. Int. Conf. 3D Vision (3DV)*, 2016.
18) PASCAL VOC, “The PASCAL Visual Object Classes Challenge 2012 (VOC2012) development kit,” online documentation. Accessed: Oct. 21, 2026.
19) COCO Consortium, “COCO API: PythonAPI/pycocotools/cocoeval.py,” GitHub repository, source code. Accessed: Oct. 21, 2026.

References

20) PyTorch Contributors, “torchvision.ops.nms,” *Torchvision documentation*. Accessed: Oct. 21, 2026.
21) scikit-learn Developers, “Metrics and scoring: Quantifying the quality of predictions,” *scikit-learn documentation*. Accessed: Oct. 21, 2026.
22) T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature pyramid networks for object detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017.



The End

Thank you for your attention!

Email: chchan@ntnu.edu.tw

Website: https://toodou.github.io/